

LABORATORIO

Pines Digitales en el Microcontrolador ATmega2560

preparado por Prof. Antonio Russoniello

1. Introducción

En este laboratorio introductorio, utilizaremos el microcontrolador ATmega2560 que se ha convertido, junto al ATmega328p y otros microcontroladores, en un estándar defacto en el mundo “maker” y especialmente para aquellos que se inician en el área de la programación de microcontroladores, es una excelente forma de introducirse en el área del Internet de las Cosas.

En este laboratorio daremos los pasos iniciales para comprender las potencialidades de la placa y sus posibles usos en el área del Internet de las Cosas.

2. ATmega 2560

La placa cuenta con las siguientes características generales:

- Microcontrolador de 8 bits.
- Contiene un reloj de 16 MHz
- Memoria de 256 KB Flash (programa), 8 KB SRAM, 4 KB EEPROM.
- Posee 54 pines digitales de entrada-salida de los cuales 15 pines pueden ser usados como salida PWM.
- Soporta 16 entradas analógicas con convertidor ADC de 10 bits.
- Incluye 6 temporizadores/contadores
- Ofrece 4 puertos para comunicación serial UART por hardware y soporta comunicación SPI (D50, D51, D52, D53) e I2C (pines D20 y D21).
- Voltaje de operación de 4,5V a 5,5V

Límites de corriente

Por cada pin Vcc/GND el límite absoluto es de 200 mA.

El chip tiene un límite total combinado que se sitúa en torno a los 800 mA, sin embargo, esta corriente debe estar distribuida entre diferentes puertos para no sobrecargar las pistas internas.

Por cada pin de E/S digital individual, se tiene un máximo absoluto de 40 mA, aunque se recomienda no exceder los 20 mA para operación continua y confiable.

2.1. Detalle de Pines

En la Figura 1 se muestra el detalle de los pines que conforman la placa, note que hay pines que poseen varias funcionalidades y que para acceder a una en específico debe indicarse en el código.

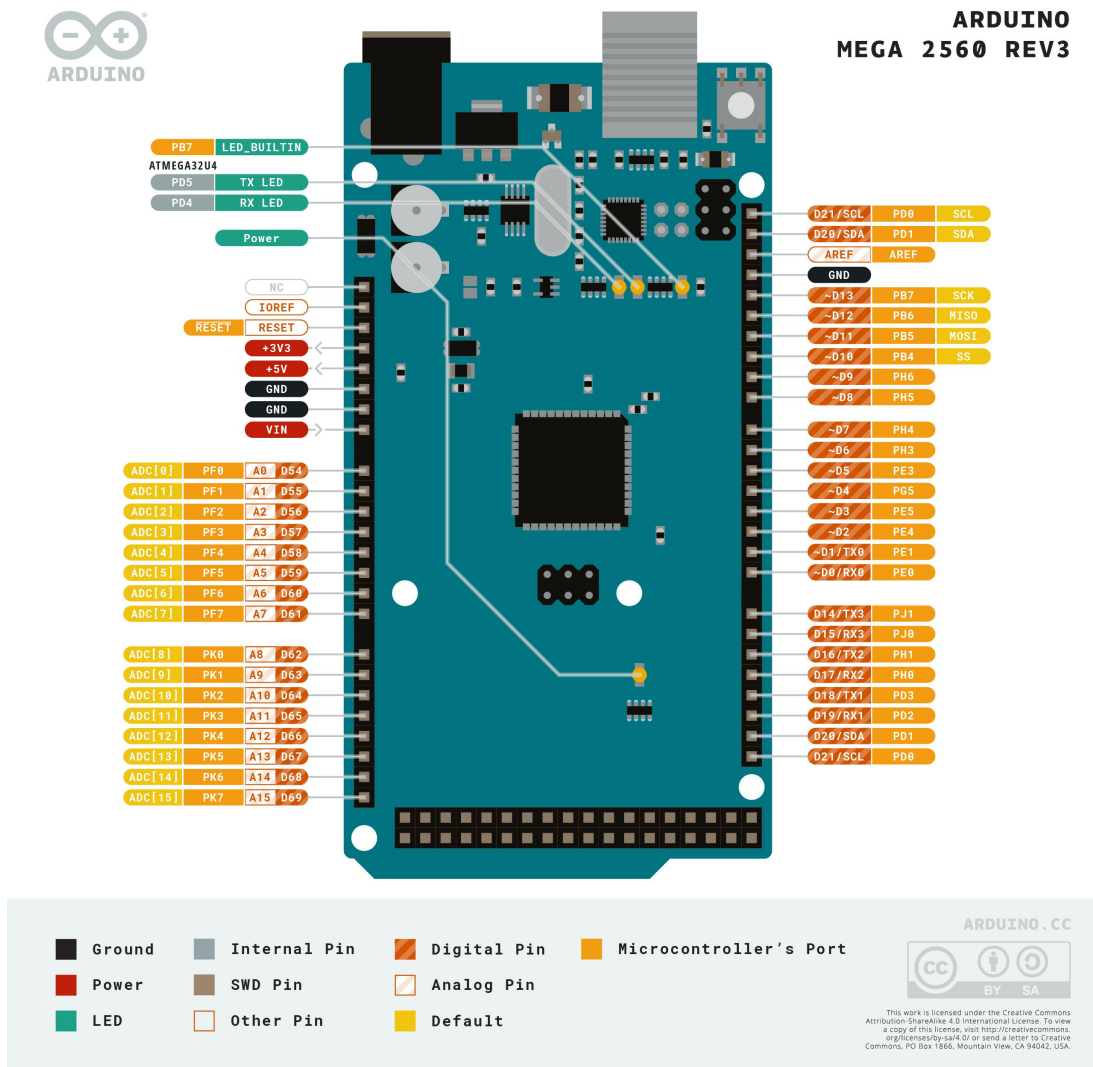


Figura 1: Detalles del Arduino Mega 2560

2.2. Administración de Memoria

Debido a que generalmente las placas utilizadas para los desarrollos cuentan con espacio de memoria reducido es importante optimizar la misma, para lograr eso se debe considerar el uso eficiente de almacenamiento de los datos considerando los distintos tipos de memoria soportados por las placas y la capacidad para cada tipo.

A continuación se mencionan los aspectos más relevantes de este tema, enfocándonos en los microcontroladores ATmega328 y ATmega2560. Empezaremos por dar una breve descripción de los tipos de memoria disponibles:

- **SRAM** (Static Random Access Memory): memoria de tipo volátil. Es el espacio de memoria donde las placas almacenan las variables utilizadas por el código en ejecución.
- **EEPROM** (Electrically Erasable Programmable Read-Only Memory): es un tipo de memoria no-volátil, este espacio de memoria puede ser usado por los desarrolladores para almacenar datos de manera permanente y borrarlos de manera explícita en caso de ser necesario. .
- **Flash**: tipo de memoria no-volátil. Es el espacio de memoria para almacenar el programa de ejecución, a veces referida como espacio de memoria de programa. En esta memoria se almacena además el bootloader o cargador de los programas que permite subir al microcontrolador el código ya compilado que será ejecutado.

En el Cuadro 1 se muestra una comparación entre las capacidades de cada tipo de memoria disponibles para el microcontrolador ATmega328 (Arduino UNO) y el ATmega2560 (Arduino MEGA 2560).

Memoria	ATmega328	ATmega2560
Flash	32 KBytes (512 bytes usados por el bootloader)	256 KBytes (8 KB usados por el bootloader)
SRAM	2 KBytes	8 KBytes
EEPROM	1 KBytes	4 KBytes

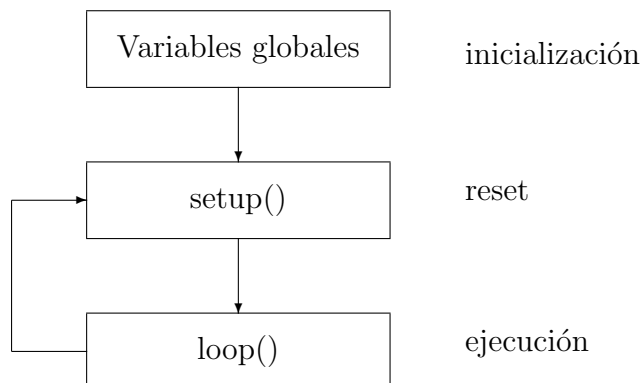
Cuadro 1: Memoria ATmega328 vs ATmega2560

Para el uso eficiente de la memoria se sugiere tomar las siguientes precauciones:

- Ejecutar todo el pre y post-procesamiento de los datos fuera de la placa, con esto aseguramos que la manipulación de los datos sea realizada por equipos con suficiente poder de cómputo y descargamos a la placa de la tarea de cómputo extremo.
- Para el uso de tablas o arreglos extensos, usar el tipo de dato más pequeño necesario para el almacenamiento de datos, por ejemplo, un *int* requiere dos bytes mientras que un tipo *byte* solo un byte.
- Si no se requiere modificar string o datos mientras el código esté en ejecución, pueden almacenarse en la memoria del programa (memoria flash) y no en la SRAM que posee un espacio de memoria reducido y mucho menor que el de la SRAM. Esto puede lograrse en las placas Arduino utilizando la instrucción `PROGMEM` disponible en la librería “`pgmspace.h`”.

3. Estructura del Código

La programación de la placa se realizará usando el IDE suministrado por Arduino, funciona bajo el entorno de sketch, y utiliza un esquema base donde en la cabecera del programa se indican las variables globales, luego una sección de inicialización (setup) y posteriormente la sección de ejecución (loop) de instrucciones. Considere que la sección de ejecución es un loop infinito que una vez alcance la última línea volverá a ejecutar la primera dentro del ciclo de ejecución.



4. Lista de Componentes

A continuación se muestran los componentes y las cantidades requeridas, por cada equipo de trabajo, para las actividades que serán ejecutadas en el laboratorio:

- Protoboard (1)
- Placa ATmega2560 (1)
- Led rojo (1)
- Led verde (1)
- Led amarillo (1)
- Resistencias de protección $220\ \Omega$ ó $330\ \Omega$ (3)
- Pulsadores (1)
- Jumpers tipo M-M

Para las actividades, el PC de cada mesa de trabajo contará con las siguientes herramientas de software:

- IDE Arduino
- Software de diseño de circuitos Fritzing

5. Actividades

Siga los pasos indicados a continuación. Para cada paso agregue en su informe capturas de pantalla que muestren el resultado correspondiente.

Conectar, Programar y Compilar:

- 5.1. Ejecutar el Arduino IDE. Investigue acerca de los pasos que debe seguir para seleccionar la placa correcta y el puerto de comunicación, indique cuál es el puerto seleccionado. Explique cómo se realiza la comunicación entre su PC y la placa Arduino. ¿Qué tipo de comunicación es utilizada? **(1 punto)**
- 5.2. Compile el sketch `lab1-1.ino` Explique qué hace el programa. **(1 punto)**
- 5.3. Compile el sketch `lab1-2.ino` y suba a la placa. Explique qué cambio nota con respecto al código del paso anterior. Modifique el sketch `lab1-2.ino` para que el led de la placa permanezca encendido por 5 segundos y apagado por 2 segundos. **(2 puntos)**
- 5.4. Cierre el IDE, desconecte la placa Arduino del cable USB, espere unos 20 segundos, vuelva a conectar la placa al cable ¿qué ocurre con el LED?, oprima el botón de Reset de la placa ¿nota algún cambio? Explique si el código compilado y subido a la placa, una vez reseteada la placa se pierde o es mantenido en memoria. Indique que tipo memoria está siendo utilizada para esta experiencia. **(2 puntos)**
- 5.5. Compile `lab1-3.ino` y `lab1-4.ino`. ¿Qué diferencias observa con respecto a `lab1-2.ino`? **(1 punto)**
- 5.6. Compile `lab1-5.ino` y `lab1-6.ino`. Intente modificar el valor de `led` dentro del ciclo `loop()` en ambos casos, explique el resultado obtenido. **(1 punto)**

Control de Pines Digitales:

- 5.7. Utilizando el protoboard construya un circuito (Figura 2) con una resistencia de protección ($220\ \Omega$) conectado a un led y encienda y apague el led cada 2 segundos, utilice para la conexión a la placa el pin digital 7. Muestre el código utilizado y el circuito. **(2 puntos)**

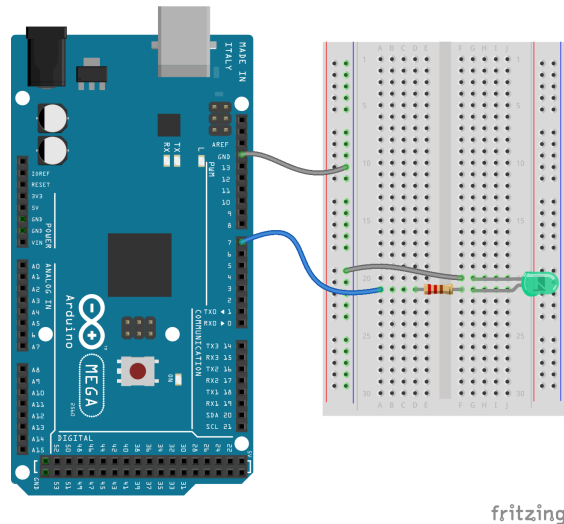


Figura 2: Circuito Led

- 5.8. Modifique el circuito anterior conectando un pulsador en el **pin digital 2** (Figura 3) y modifique su código para que el led encienda y apague cada vez que se presione el **pulsador**. (2 puntos)

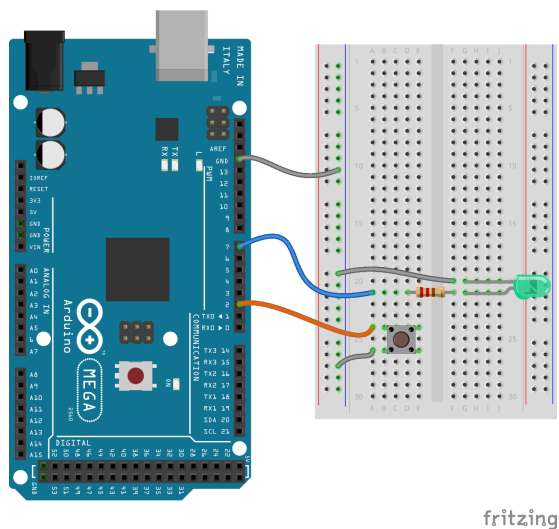


Figura 3: Circuito Pulsador y Led

- 5.9. Programe un semáforo con tres leds (rojo, verde y amarillo) conectados a tres pines digitales con los siguientes tiempos: 4 segundos para el rojo, 1 segundo para el amarillo, 5 segundos para el verde. Construya el circuito correspondiente (ver Figura 4 como referencia) recuerde que debe incorporar las resistencias de protección. Ejecute el código en el Arduino y verifique su operación. Construya el circuito utilizando para ello la herramienta **Fritzing**. (8 puntos)

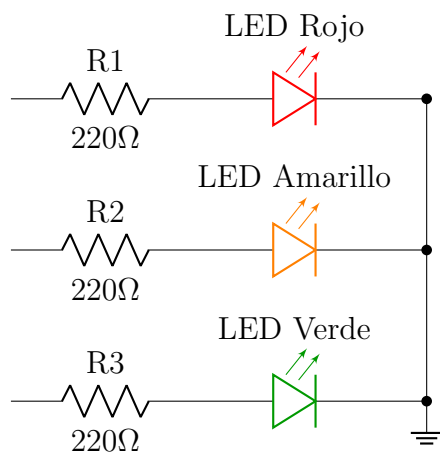


Figura 4: Circuito para el semáforo

§

Total de puntos: 20